RV32IM SoC

Core Verification Reference

Functional Verification Plan, Methodology & Results

1. Overview

Verification is performed in two distinct phases: an initial self-directed register-dump comparison using the Spike ISA simulator, followed by a formal RISC-V compatibility campaign using RISCOF and the official RISC-V International architectural tests.

The goal of Phase 1 is to establish basic architectural correctness across a representative set of hand-written and compiler-generated C programs. The goal of Phase 2 is to achieve official RISC-V compliance demonstrating that the core conforms to the RV32IM unprivileged specification and to measure functional coverage across all mandatory instruction groups.

All verification is performed against Spike (RISC-V ISA Reference Simulator) as the golden model. Spike is the official ISA reference implementation maintained by RISC-V International and is the mandated golden model for RISCOF-based compliance testing.

2. Verification Architecture

The end-to-end verification flow is organised into two phases with a shared golden model. Both phases use Spike as the reference, ensuring consistent architectural ground truth across all test campaigns.

2.1 Golden Model — Spike ISA Simulator

Spike is the reference implementation of the RISC-V ISA maintained by RISC-V International. It is used in both phases of this verification campaign as the authoritative architectural oracle.

- Spike simulates the full RV32IM ISA at the instruction-retire level.
- Register state is observable after any instruction via Spike's interactive or batch logging modes.
- Spike is configured for the same privilege level (M-mode only) and memory layout as the DUT.
- The Spike base address is set to 0x8000 during Phase 1 standalone runs and reconfigured to match RISCOF test harness expectations during Phase 2.

2.2 Design Under Test (DUT)

Attribute	Value
ISA	RV32I
Pipeline Type	In-order, Single Cycle
Simulator	Icarus Verilog (iverilog) + VVP
Register Dump Format	Plain text, one register per line: reg_name = hex_value
Spike Base Address (Phase 1)	0x8000
RTL Base Address	0x0 (Boot ROM)

3. Phase 1 — Spike Register Dump Comparison

3.1 Phase 1 Objectives

Phase 1 establishes baseline correctness for the DUT core before committing to the full RISCOF compliance campaign. The objectives are:

- Confirm that the 5-stage pipeline correctly implements the RV32IM instruction set for representative programs.
- Identify and fix pipeline hazard bugs (RAW data hazards, control hazards, load-use hazards) early in the design cycle.
- Validate the multiply/divide unit (M extension) on both positive and negative operand combinations.
- Establish a known-good RTL baseline for the Phase 2 RISCOF campaign.

3.2 Verification Flow

The Phase 1 flow compiles C test programs to two separate binary images — one linked for the RTL base address, one for Spike — simulates both, captures register dumps, and runs a normalised comparison script to produce a PASS or FAIL result.

Phase 1 Verification Flow

```
C Test Program (.c)
          |
          ├─── riscv32-gcc -march=rv32im  (RTLbase=0x0)  ──────→ iverilog/vvp
simulation
          |                                                              |
          |                                                       RTL Register Dump
          └─── riscv32-gcc -march=rv32im  (SpikeBase=0x8000) ─→ Spike ISA
simulation
                                                                         |
                                                                 Spike Register Dump
                                                                         |
                                                              Normalise + Compare
Script
                                                                         |
                                                                  PASS  /  FAIL
```



3.4 Register Comparison Methodology

At the end of each simulation, all 32 architectural registers (x0 through x31) are dumped from both the DUT and Spike. A Python comparison script normalises both dumps to a canonical format before diffing, to avoid false failures due to whitespace or formatting differences between the two simulators.

3.4.1 DUT Register Dump Format

```
abhi@Abhi-ubuntu:~/Desktop/RISCFORGE_1TOPS/Testing_Setup$ make test
============================================================
  PROGRAM HALTED at PC = 0x0000000c  (cycle 24)
  Halt instruction: 0x0000006f
============================================================

                RISC-V REGISTER FILE DUMP
Reg        ABI Name        Hex Value          Decimal
x0         zero            0x00000000         0
x1         ra              0x0000000c         12
x2         sp              0x00000ffc         4092
x3         gp              0x00000000         0
x4         tp              0x00000000         0
x5         t0              0x00000000         0
x6         t1              0x00000000         0
x7         t2              0x00000000         0
x8         s0/fp           0x00000000         0
x9         s1              0x00000000         0
x10        a0  ◄RET        0x0000000d         13
x11        a1              0x00000000         0
x12        a2              0x00000000         0
x13        a3              0x00000000         0
x14        a4              0x00000006         6
x15        a5              0x0000000d         13
x16        a6              0x00000000         0
x17        a7              0x00000000         0
x18        s2              0x00000000         0
x19        s3              0x00000000         0
x20        s4              0x00000000         0
x21        s5              0x00000000         0
x22        s6              0x00000000         0
x23        s7              0x00000000         0
x24        s8              0x00000000         0
x25        s9              0x00000000         0
x26        s10             0x00000000         0
x27        s11             0x00000000         0
x28        t3              0x00000000         0
x29        t4              0x00000000         0
x30        t5              0x00000000         0
x31        t6              0x00000000         0

  >>> Return value (a0/x10) = 13 (0x0000000d) <<<
```

3.4.2 Spike Register Dump Format

Register	ABI	Spike (Gold)	RTL (DUT)	Status
x0	zero	0x00000000	0x00000000	✓
x1	ra	0x8000000c	0x0000000c	✓ adj
x2	sp	0x80000ffc	0x00000ffc	✓ adj
x3	gp	0x00000000	0x00000000	✓
x4	tp	0x00000000	0x00000000	✓
x5	t0	0x80000000	0x00000000	~ skip
x6	t1	0x00000000	0x00000000	✓
x7	t2	0x00000000	0x00000000	✓
x8	s0/fp	0x00000000	0x00000000	✓
x9	s1	0x00000000	0x00000000	✓
x10	a0	0x0000000d	0x0000000d	✓
x11	a1	0x00001020	0x00000000	~ skip
x12	a2	0x00000000	0x00000000	✓
x13	a3	0x00000000	0x00000000	✓
x14	a4	0x00000006	0x00000006	✓
x15	a5	0x0000000d	0x0000000d	✓
x16	a6	--	0x00000000	?
x17	a7	--	0x00000000	?
x18	s2	--	0x00000000	?
x19	s3	--	0x00000000	?
x20	s4	--	0x00000000	?
x21	s5	--	0x00000000	?
x22	s6	--	0x00000000	?
x23	s7	--	0x00000000	?
x24	s8	--	0x00000000	?
x25	s9	--	0x00000000	?
x26	s10	--	0x00000000	?
x27	s11	--	0x00000000	?
x28	t3	--	0x00000000	?
x29	t4	--	0x00000000	?
x30	t5	--	0x00000000	?
x31	t6	--	0x00000000	?

✅ PASS — 14/14 compared registers match (2 skipped)
Return value (a0): Spike=0x0000000d RTL=0x0000000d

3.5 Phase 1 Results Summary

The table below records the final pass/fail status for each test program after RTL debug and pipeline fix iterations. All 15 programs achieve PASS status in the final RTL revision committed to the Phase 2 baseline.

Test Program	First Run	Final Run	Bugs Found & Fixed
arith_basic.c	PASS	PASS	None
logic_ops.c	PASS	PASS	None
shift_ops.c	FAIL	PASS	SRA sign-extension in EX stage incorrect for negative operands
compare_ops.c	PASS	PASS	None
branch_test.c	FAIL	PASS	BGE/BLTU flush target computed one cycle late — PC+4 issued before redirect
jump_test.c	PASS	PASS	None
load_store.c	FAIL	PASS	LH sign-extension: upper 16 bits not replicated on negative halfword load
csr_ops.c	PASS	PASS	None
multiply.c	FAIL	PASS	MULHSU: mixed signed/unsigned product incorrect for negative rs1
divide.c	FAIL	PASS	DIV overflow case (0x80000000 / 0xFFFFFFFF): result must be 0x80000000 per spec
hazard_raw.c	FAIL	PASS	Forwarding from MEM-to-EX stage missing for load-use sequence
hazard_control.c	FAIL	PASS	Pipeline flush did not squash instruction in ID stage — stale instruction leaked
fibonacci.c	PASS	PASS	None (after upstream fixes)
bubble_sort.c	PASS	PASS	None (after upstream fixes)
matrix_mult.c	PASS	PASS	None (after upstream fixes)

Phase 1 Summary: 15 / 15 test programs PASS. 6 pipeline bugs identified and fixed during Phase 1. RTL baseline is now frozen for Phase 2.

4. Phase 2 — RISCOF Compliance Campaign

4.1 What is RISCOF?

RISCOF (RISC-V Compatibility Framework) is the official testing framework developed and maintained by RISC-V International. It provides a standardised, automated infrastructure for determining whether an implementation conforms to the RISC-V architectural specification.

- RISCOF manages the execution of the official RISC-V Architectural Test Suite (RVTS) against both the DUT and the reference model (Spike).
- The framework compares the Signature region — a defined memory region written by the test between DUT and reference. If the signatures match for all tests in an extension group, the implementation is considered compliant for that group.
- RISCOF generates a compliance report (HTML + YAML) that lists PASS/FAIL per test case and per extension group.
- RISCOF compliance is a prerequisite for applying for the RISC-V Compatibility Certificate from RISC-V International.

4.2 RISCOF Framework Architecture

RISCOF operates through a plugin system where the DUT team provides a Python plugin class that knows how to compile and run a test on the specific implementation. The framework then orchestrates test discovery, compilation, simulation, and signature comparison automatically.

Component	Role in Campaign	Implementation
RISCOF CLI	Test orchestration, report generation	Python package — pip install riscof
RISCV-ARCH-TEST	Official test suite from RISC-V International	Git submodule: riscv-arch-test (main branch)
Spike Plugin	Reference model plugin — supplied with RISCOF	riscof_spike Python class, Spike binary
DUT Plugin	Custom plugin for iverilog/vvp simulation of RV32IM core	riscof_rv32im Python class (custom)
riscv-gnu-toolchain	Compilation of test ELFs for RV32IM target	riscv32-unknown-elf-gcc -march=rv32im -mabi=ilp32
Signature Region	Memory area written by each test to record results	Defined at 0x80000000 in test linker script
YAML Config	Declares implemented ISA extensions and hart count	config.ini + isa.yaml (hand-authored)

4.3 Architectural Test Suite Structure

The RISC-V Architectural Test Suite (RVTS) is maintained by RISC-V International in the riscv-arch-test GitHub repository. Each test is a self-contained RISC-V assembly file that exercises a specific instruction or edge case and writes its result to the Signature memory region.

Test Group	Extension	Test Count (RV32)	Coverage Target
rv32i	RV32I Base Integer	52 tests	All 47 base instructions, edge cases, hazard sequences
rv32m	RV32M Multiply/Divide	16 tests	All 8 M-extension instructions, signed/unsigned boundary cases
Zicsr	CSR Instructions	6 tests	CSRRW, CSRRS, CSRRC and immediate variants
Zifencei	Instruction-Fetch Fence	1 test	FENCE.I instruction, I-cache coherence (NOP on this core)
privilege	M-Mode Exception Handling	8 tests	Illegal instruction, misaligned, ecall, ebreak

Verification Plan & Current Status

Milestone	Activity	Status	Notes
M1	RTL baseline + testbench setup	Complete	iverilog simulation environment, register dump mechanism
M2	Phase 1: Hand-written C test suite (15 programs)	Complete	All 15 programs PASS, 7 bugs found and fixed
M3	Phase 2: RISCOF environment setup	In Progress	Plugin written, isa.yaml configured, test suite cloned
M4	Phase 2: RV32I compliance run	In Progress	52 tests being executed against current RTL
M5	Phase 2: RV32M compliance run	In Progress	16 tests queued pending RV32I clean pass
M6	Phase 2: Zicsr + Zifencei compliance run	Planned	After M5 completion
M7	Full compliance report generation	Planned	HTML report for project record
M8	Functional coverage analysis	Planned	Coverage model completion and gap analysis
M9	Formal compliance certificate application	Future	Pending successful RISCOF campaign

Tools & Environment

Tool / Component	Version	Purpose
Icarus Verilog (iverilog)	v11.0	RTL simulation of DUT
VVP Runtime	v11.0	VCD generation, memory dump, $finish
Spike ISA Simulator	v1.1.1-dev	Golden model (Phase 1 + RISCOF reference plugin)
riscv-gnu-toolchain	2024.04 (rv32im)	Compilation: riscv32-unknown-elf-gcc -march=rv32im -mabi=ilp32
RISCOF	v0.23.0	Compliance framework — pip install riscof
riscv-arch-test	main branch (March 2026)	Official RISC-V Architectural Test Suite
Python	v3.11	RISCOF plugins, comparison scripts
GTKWave	v3.3.116	Waveform inspection for debug
Git	v2.43	Version control — RTL, tests, scripts

Known Limitations & Future Work

1. RISCOF campaign is in progress. Full compliance results will be added to a subsequent revision of this document once all test groups complete.
2. WFI instruction is not covered by the official RISC-V Architectural Test Suite and is verified only through the boot ROM sequence test in Phase 1.
3. Privilege-mode exception tests (ECALL, EBREAK, illegal instruction) depend on a correctly implemented trap handler in the simulation testbench. This is under development.
4. No formal verification (SVA property checking or model checking) has been performed in this revision. Formal properties for the pipeline hazard logic are planned for a future revision.